Google ADK

1 基础介绍

谷歌Agent Development Kit （ADK）是谷歌公司推出的一款开源AI智能体开发框架。2025年4月9日，在拉斯维加斯举办的Google Cloud Next 2025大会上，谷歌正式发布了ADK。这是一款开源的、支持Python、Go、TypeScript和Jav等多种语言的AI智能体开发框架，旨在简化多智能体系统的构建、管理与部署。其核心特性包括模块化架构、代码优先的开发方式、兼容Gemini等大语言模型以及MCP协议、内置的工作流编排与动态路由能力、多模态交互支持、与谷歌云服务的深度集成，并支持容器化部署和长短期记忆功能。

ADK采用事件驱动的运行时架构，以Runner为核心组件。通过事件循环处理用户请求、模型调用与工具执行，支持复杂的多步推理与状态管理。该架构将智能体、工具和持久化状态整合为统一应用，为构建生产级AI应用提供了框架。该框架适用于复杂任务调度等场景，如零售动态定价、电动车充电站选址等。

设计初衷是让智能体开发流程可控，采用代码优先的开发方式，通过代码定义智能体行为，提供了较强的控制、可测试性和版本控制。

2 核心技术（架构设计）

ADK 作为一个功能强大的开放源代码框架，为开发者提供了灵活的基础，其核心组件，包括但不限于代理架构、模型集成、工具生态以及运行环境等。https://github.com/google/adk-python

核心特性

- **工作流运行时（Workflow Runtime）**：一个基于图的执行引擎，用于为智能体应用编排确定性的执行流程，支持路由、扇出/扇入（fan-out/fan-in）、循环、重试、状态管理、动态节点、人工介入（human-in-the-loop）以及嵌套工作流。
- **任务 API（Task API）**：结构化的智能体间委派机制，支持多轮任务模式、单轮受控输出、混合委派模式、人工介入，以及将任务智能体用作工作流节点。
- **模块化多智能体系统（Modular Multi-Agent Systems）**：通过把多个专用智能体组合成灵活的层级结构，来设计可扩展的应用。
- **丰富的工具生态（Rich Tool Ecosystem）**：可使用预置工具、自定义函数、OpenAPI 规范、MCP 工具，也可集成已有工具，为智能体赋予多样化的能力，并与 Google 生态深度集成。
- **代码优先开发（Code-First Development）**：直接用 Python 定义智能体逻辑、工具与编排，从而获得最大的灵活性、可测试性和版本管理能力。
- **智能体配置（Agent Config）**：无需写代码即可构建智能体。参见 ⊕ Agent Config 功能。
- **工具确认（Tool Confirmation）**：一套工具确认流程（HITL，即人工介入），可通过显式确认和自定义输入为工具执行把关。
- **随处部署（Deploy Anywhere）**：轻松将智能体容器化并部署到 Cloud Run，或借助 Vertex AI Agent Engine 实现无缝扩缩容。

代码块

```
from google.adk import Agent, Context, Event, Runner, Workflow

# ① 入口层：谁来跑
from google.adk import Runner                    # 执行引擎入口
```

```
from google.adk.apps import App, ResumabilityConfig  # 打包/可恢复配置

# ② 编排层：怎么连
from google.adk import Workflow                      # 图执行引擎（2.x 核心新增）
-- 一个基于图的执行引擎，用于为智能体应用编排确定性的执行流程，支持路由、扇出/扇入（fan-out/fan-in）、
-- 循环、重试、状态管理、动态节点、人工介入（human-in-the-loop）以及嵌套工作流。
from google.adk.workflow import (
    Node, node, FunctionNode, JoinNode, Edge,
    START, DEFAULT_ROUTE, RetryConfig, NodeTimeoutError,
)
# ③ 智能层：想什么
from google.adk.agents import LlmAgent, BaseAgent, ManagedAgent, RunConfig
from google.adk.models import Gemini, LiteLlm, Claude, BaseLlm, LLMRegistry
# ADK默认的大模型调用工具，LiteLlm
from google.adk.planners import BasePlanner, BuiltInPlanner, PlanReActPlanner
from google.adk.code_executors import ...            # 代码执行器

# ④ 能力层：能干什么
from google.adk.tools import FunctionTool, AgentTool, MCPToolset, google_search, ToolContext
from google.adk.auth import ...                      # 凭据/OAuth
from google.adk.integrations import ...              # 第三方模型（OpenAI/OCI）

# ⑤ 状态层：记住什么（全是可替换的 Service）
from google.adk.sessions import (
    Session, State, BaseSessionService,              # 会话内状态
    InMemorySessionService, DatabaseSessionService, VertexAiSessionService,
)
from google.adk.memory import BaseMemoryService, InMemoryMemoryService, SqliteMemoryService  # 跨会话长期记忆
from google.adk.artifacts import InMemoryArtifactService, FileArtifactService, GcsArtifactService  # 文件产物

# ⑥ 信息层：一切结果的载体
from google.adk.events import Event, EventActions, RequestInput

# 一句话概括：
Runner 组装上下文 → Agent 调模型 → 模型要工具就执行工具再回模型 → 每一步都吐 Event → Event 回写 Session。

上面的每一块 都要单开一章学习介绍。
```

代码块

```
执行顺序
```

```
 ① 定义 Agent（纯声明，此时不跑任何东西）
from google.adk.agents import LlmAgent
agent = LlmAgent(name="helper", model="gemini-2.5-flash", instruction="...",
tools=[...])

# ② 选 Session 服务（状态存哪：内存 / 数据库 / Vertex）
from google.adk.sessions import InMemorySessionService
session_service = InMemorySessionService()

# ③ 创建 Runner —— 把 agent 和所有 service 绑在一起，这是执行引擎
from google.adk.runners import Runner
runner = Runner(agent=agent, app_name="demo",
                session_service=session_service)   # session_service 是唯一必填项

# ④ 建会话（或用 auto_create_session=True 让 Runner 自动建）
session = await session_service.create_session(app_name="demo", user_id="u1")

# ⑤ 组装用户消息
from google.genai import types
msg = types.Content(role="user", parts=[types.Part(text="你好")])

# ⑥ 开跑 —— 拿到的是一条事件流，开跑入口唯一
async for event in runner.run_async(user_id="u1", session_id=session.id,
new_message=msg):
    print(event.author, event.content)

# ⑦ 收尾（关闭 MCP 工具集等）
await runner.close()
```

2.1 Agent

2.2 Runner

2.3 Workflow

2.4 Context

2.5 Session

2.6 Event

2.7 tools

2.8 artifacts

3 快速入门

场景一：旅游规划项目

见代码

场景二：水单认领智能体

4 ADK 总结



ADK 不只是一个写 agent 的框架，它是一套「生产级 Agent 运行时」，ADK 让你的 agent 像一套系统一样运行。

- 解放创造力：路由、扇出/扇入、循环、嵌套、重试、状态管理、人工介入、断点恢复——这些在别处要你自己搭的工程脏活，ADK 全部内置。你只需要专注真正有价值的部分：这个 agent 该怎么思考、该用哪些工具、领域知识怎么沉淀。
- 降低门槛：一个 `LlmAgent(name, model, instruction, tools)` 就是完整可跑的 agent；目录按约定摆好（`agent.py` + `root_agent`），一条 `adk web .` 就有可视化调试界面；连代码都不想写，还能用 YAML 声明式配置。